

1 React `useReducer` – Complete Guide (Nepali + English Mix)

2 Introduction

`useReducer` React ko built-in Hook ho jun **complex state management** ko lagi use garinx.

Simple state manage garna `useState` use garxau.

Tara euta state lai **dherai different ways bata update garna parne** bhayo vane `useReducer` better choice ho.

3 Why Do We Need `useReducer`?

Suppose counter xa.

```
const [count, setCount] = useState(0);
```

Increase garna

```
setCount(count + 1);
```

Yo ekdam simple xa.

Aba Login Page banaudai xau.

```
const [loading, setLoading] = useState(false);  
const [user, setUser] = useState(null);  
const [error, setError] = useState("");  
const [success, setSuccess] = useState(false);
```

Aba Shopping Cart.

```
const [cart, setCart] = useState([]);  
const [total, setTotal] = useState(0);  
const [discount, setDiscount] = useState(0);  
const [shipping, setShipping] = useState(0);  
const [coupon, setCoupon] = useState("");
```

Aba state dherai bhayo.

Update garne function pani dherai bhayo.

Code alik messy huna thalxa.

Yestai situation ma **useReducer** use garinx.

4 Real-Life Example (TV Remote)

Imagine timro ghar ko TV.

TV ko volume change garna TV kholera wire change gardainau.

Instead, **Remote** use garxau.

Remote bata command janxa.

Power

Volume +

Volume -

Mute

Channel +

TV le command receive garxa ani decide garxa k garne.

`useReducer` pani exactly yei concept ma kaam garxa.

Button Click

↓

`dispatch()`

↓

Reducer

↓

New State

↓

React UI Update

5 useReducer k ho?

Definition

`useReducer` is a React Hook used to manage complex state updates using actions instead of directly updating the state.

Simple Nepali ma,

`setState()` directly use nagari,

hami **Action** pathauxau.

Reducer le tyo action herera state kasari update garne decide garxa.

6 Syntax

```
const [state, dispatch] = useReducer(reducer, initialState);
```

Yaha 3 ota important kura xa.

Initial State

↓

Reducer

↓

Dispatch

7 1. Initial State

State ko starting value.

Example

```
const initialState = {  
  count: 0  
};
```

Suru ma

```
count = 0
```

8 2. Reducer Function

Reducer euta normal JavaScript function ho.

Yo function le 2 ota parameter linxa.

```
function reducer(state, action){  
}
```

9 state

Current state.

10 action

User le dispatch gareko command.

Reducer ko kaam

- Action receive garne
- K change garne decide garne
- New state return garne

11 3. Dispatch Function

Dispatch ko kaam reducer lai action pathaune ho.

Example

```
dispatch({  
  type: "increment"  
});
```

Yasko meaning

Hello Reducer,

Please increment the count.

12 Overall Flow

User Click

↓

dispatch()

↓

Reducer()

↓

New State Return

↓

React Re-render

↓

Updated UI

13 Counter Example

14 Step 1

Initial State

```
const initialState = {  
  count: 0  
};
```

15 Step 2

Reducer

```
function reducer(state, action){  
  switch(action.type){  
    case "increment":  
      return {  
        count: state.count + 1  
      };  
    default:
```

```
        return state;
    }
}
```

Yaha

increment action ayo?

Yes

↓

count + 1

↓

new state return

16 Step 3

Hook Use Garne

```
const [state, dispatch] = useReducer(
  reducer,
  initialState
);
```

state ma current value basxa.

dispatch action pathauna use hunxa.

17 Step 4

Button

```
<button
  onClick={() =>
    dispatch({ type: "increment" })
  }
>
```

Increase

```
</button>
```

Button click hune bittikai

```
dispatch()
```

↓

```
Reducer
```

↓

```
Increment
```

↓

```
New State
```

18 Step 5

Display

```
<h1>{state.count}</h1>
```

19 Complete Code

```
import { useReducer } from "react";

const initialState = {
  count: 0
};

function reducer(state, action){
  switch(action.type){
    case "increment":
      return {
        count: state.count + 1
      };
    default:
      return state;
  }
}

export default function App(){
```

```

const [state, dispatch] = useReducer(
  reducer,
  initialState
);

return(
  <>
    <h1>{state.count}</h1>

    <button
      onClick={() =>
        dispatch({ type: "increment" })
      }
    >
      Increase
    </button>
  </>
);
}

```

20 Why `default: return state;`?

Suppose

```

dispatch({
  type: "hello"
});

```

Reducer ma

```

switch(action.type){

```

hello case xaina.

Aba React le k garne?

Old state nai firta dinxa.

```

default:
  return state;

```

Yo safety ko lagi ho.

21 Multiple Actions

Reducer le euta matra action handle gardaina.

Example

```
function reducer(state, action){  
  switch(action.type){  
    case "increment":  
      return {  
        count: state.count + 1  
      };  
    case "decrement":  
      return {  
        count: state.count - 1  
      };  
    case "reset":  
      return {  
        count: 0  
      };  
    default:  
      return state;  
  }  
}
```

Buttons

```
dispatch({ type: "increment" });  
dispatch({ type: "decrement" });  
dispatch({ type: "reset" });
```

Flow

User Click

↓

dispatch()

↓

Reducer

↓

Correct Case

↓

New State

↓

UI Update

22 Action Object

Action ma type matra hudaina.

Extra data pani pathauna milxa.

```
dispatch({
  type: "increment",
  payload: 5
});
```

Reducer

```
case "increment":

return{

  count: state.count + action.payload

}
```

Result

Count +5

Another example

```
dispatch({

  type:"setName",

  payload:"Manoj"

});
```

Reducer

```
return{  
  ...state,  
  name: action.payload  
}
```

23 Real-Life Example (Shopping Cart)

Initial State

```
const initialState = {  
  cart: []  
};
```

Button

```
dispatch({  
  type: "ADD_PRODUCT",  
  payload: product  
});
```

Reducer

```
case "ADD_PRODUCT":  
  return{  
    ...state,  
    cart: [  
      ...state.cart,  
      action.payload  
    ]  
  }  
}
```

Flow

Add Button

↓

dispatch()
↓
Reducer
↓
ADD_PRODUCT
↓
Cart Updated
↓
UI Re-render

24 useState vs useReducer

useState	useReducer
Simple state ko lagi	Complex state ko lagi
setState() use garxa	dispatch() use garxa
Direct state update	Action bata update
Small components	Large components
Easy to learn	More organized

25 useReducer vs Redux

Duitai ko purpose **State Management** ho.

Flow almost same xa.

dispatch()
↓
Reducer
↓
State Update

Difference

26 useReducer

- React ko built-in Hook
- Component level state manage garxa
- Separate package install garnu pardaina

27 Redux

- External Library ho
- Entire Application ko Global State manage garxa
- Central Store use garxa
- Large Production Applications ma dherai use hunxa

28 Industry Use Cases

useReducer dherai use hune examples:

- Login Authentication
- Shopping Cart
- Todo Application
- Multi-Step Forms
- Dashboard State
- Game State
- Wizard Forms
- Complex Form Validation

29 Important Interview Question

Q. useState ra useReducer ma difference k ho?

Answer:

useState simple state ko lagi best ho.

useReducer chai complex state management ko lagi best ho jaha same state lai dherai actions le update garna parxa.

30 Summary

- `useReducer` = Complex State Management Hook
- `initialState` = Starting state
- `reducer()` = State kasari update garne decide garne function
- `dispatch()` = Reducer lai action pathaune function
- `action.type` = Kun operation perform garne
- `action.payload` = Extra data pathaune
- `default: return state` = Unknown action aaye old state firta dinxa
- `useReducer` ko concept bujhyo bhane Redux sikna dherai easy hunxa.